

Broken Authentication

This attack is related to poor user authentication and session management.

When a system implements its authentication poorly, it's often vulnerable to attacks in this category. For example, the system can use default user names (such as "admin"), or allow weak common passwords (such as "Password1"). Other examples include Brute Force Attacks, Dictionary Attacks, Credential Stuffing, and Session Hijacking.

RockYou:-

In 2009, a company name RockYou was hacked. The attacker got their hands on over 32 million user credentials - all in plain text. The attacker then compiled a database of all the passwords, and made this list publically available. This list (named rockyou.txt) is now famous in the InfoSec community since it gives a very valuable insight in how people tend to choose their passwords. Kali Linux comes with this text file by default (found at /usr/share/wordlists/rockyou.txt.gz)

Brute Force Attacks and Dictionary Attacks

When an attacker systematically tries every possible character combination to guess passwords, this is known as a Brute Force Attack. The downside with this is that depending on the length and complexity of the passwords, it could take years to find the right one.

A variation of the Brute Force Attack is the Dictionary Attack, where instead of trying arbitrary combinations of characters, the attacker uses dictionaries. This is helpful since most passwords are words (or at least include them) and not just random characters strung together. Dictionary Attacks also tend to be faster than normal Brute Force Attacks.

Credential Stuffing: -

When user credentials get breached through hacks, they can spread online. Using breached credentials and testing them on many different websites, like social media platforms or online marketplaces, is called Credential Stuffing.

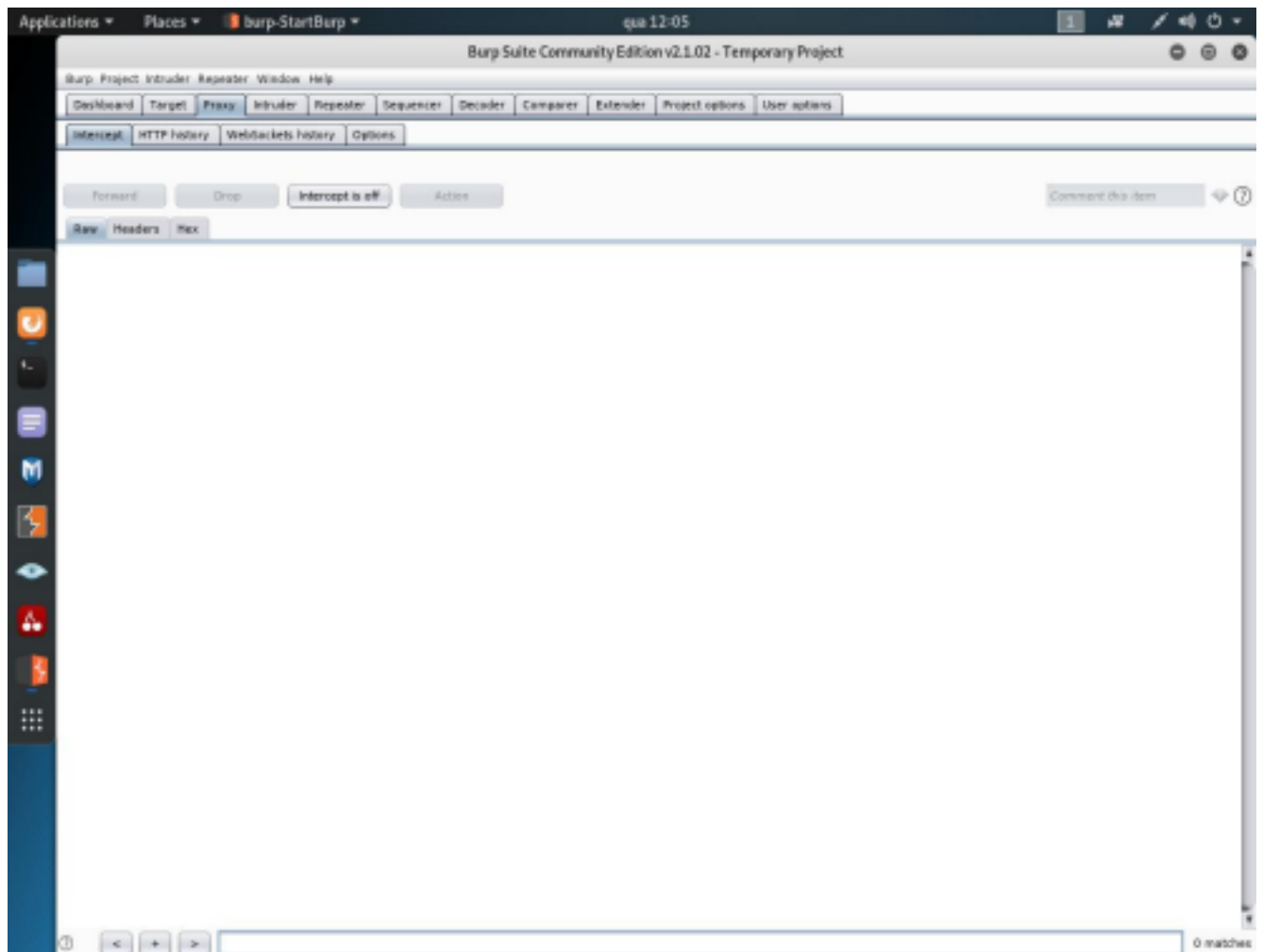
Session Hijacking: -

Since HTTP is stateless, many websites use Cookies as identification tokens. Session Hijacking, or Cookie Hijacking, is when an attacker manages to steal a Cookie and thereby gains unauthorized access to the website.

Open your browser and Burp Suite. Set the browser to use Burp:

Low

Before beginning remove the 'Intercept' from Burp:



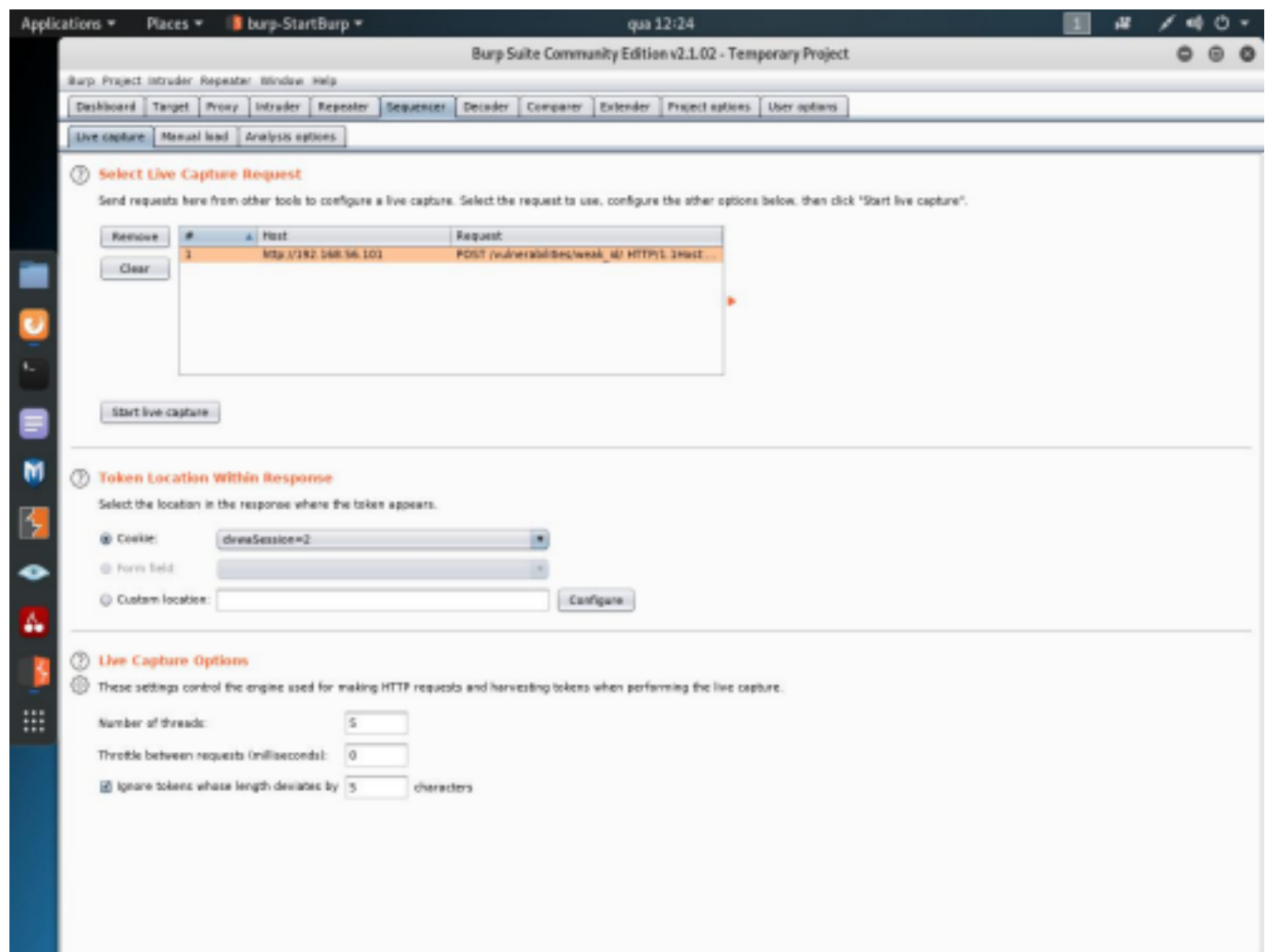
Open DVWA's page and set the security level to low. Navigate to 'Weak Session IDs':

Every time the 'Generate' button is clicked, the cookie 'dvwaSession' will receive a new value. Let's analyse how are these values created and how secure they are. Click the button and head to Burp. Click the button again and look at Burp:

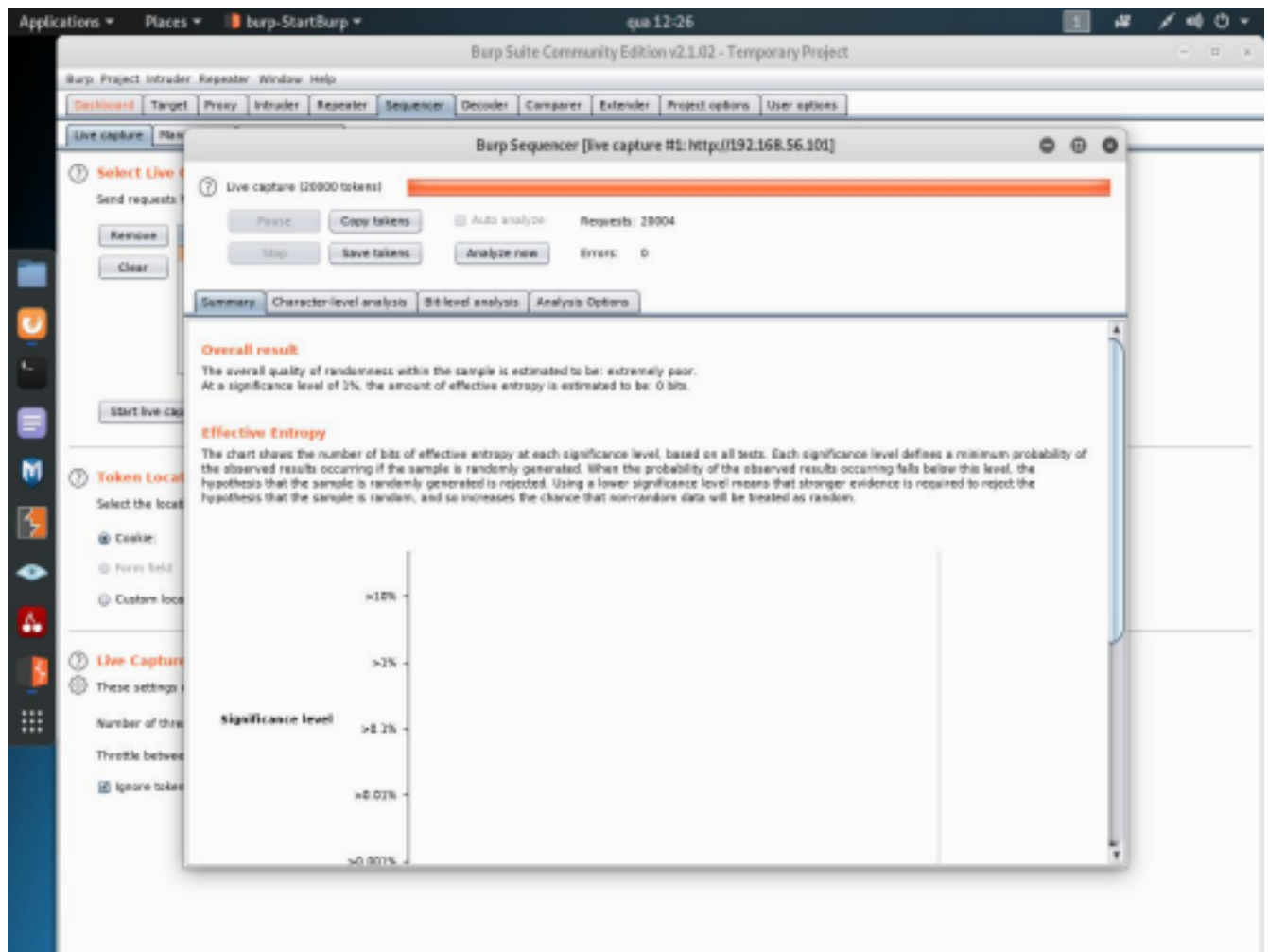
dvwaSession's first value was '1', then '2'. The next one will be '3'. There is no randomness, the values are easily predictable. This opens a Man-In-The-Middle vector of attack.

Let's use one of the functions Burp Suite has to evaluate the quality of randomness, the sequencer. Right click the request and choose 'Send to

sequencer’:



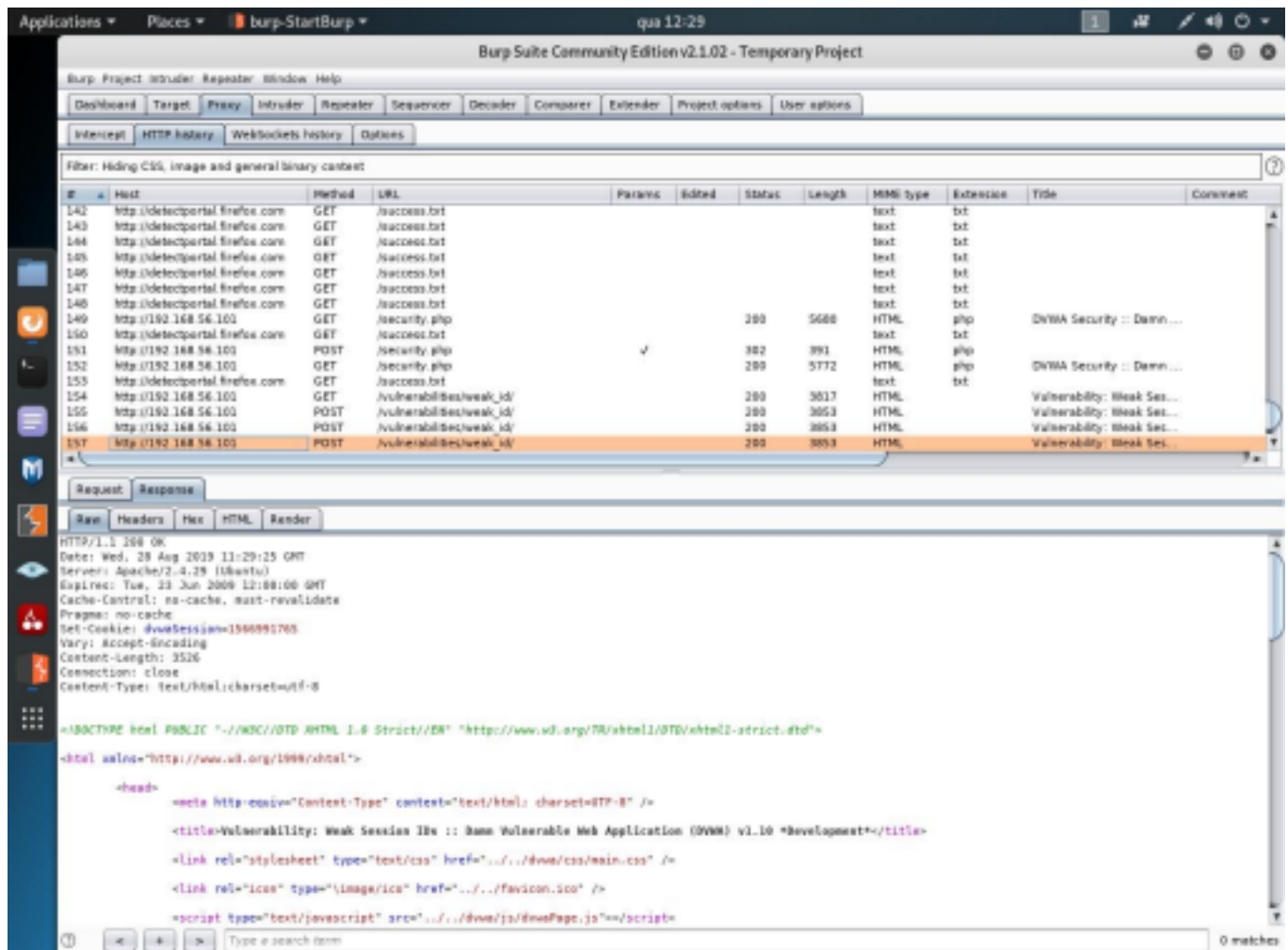
Do a start live capture and when finished click Analyse now.



The results are bad, has expected. There's no randomness in the value. Let's try the medium security level.

Medium

Clear the sequencer and set the security to medium. Go and click generate a couple of times.



dvwaSession's value seems more secure. But is it really secure? Let's use Burp's sequencer to determine that. Proceed as before to obtain an analysis.

Again, the results are bad and there is no randomness. dvwaSession's value may seem secure but it's really just the actual date in seconds.

High

Set the security to high and click a couple of times the generate button. Analyse the samples with Burp:

Burp rates the value's randomness quality as excellent. But is it? Take the value and try some conversion algorithms, like MD5.

Open a terminal window and type the command: `$`

```
findmyhash MD5 -h copy_hash_value
```

After all it's not so impregnable. Quite the opposite! Let's analyse it in Impossible.